Comparative Language Analysis

For my comparative project I chose to implement Conway's game of life in Lua, Julia, and Haskell.

All programs were tested with the same 3 seeds on the same size grids with a target frame time of 0.1 seconds

Lua

About Lua

Lua is an interpreted scripting language that runs on a virtual machine. This makes the language extremely portable at the cost of some performance overhead. We can see this when we look at the performance stats and see Lua has the highest average frame time. We also see something else interesting, Lua has the lowest memory usage of the group. This is most likely due to the fact that Lua was designed to be embedded in other languages so its runtime memory usage is very small.

Lua Developer Experience

Lua is the language I have the most experience with so from a developer experience stand point it was the easiest. The table data structure is great for representing the game of life grid, and with the __index meta method we can define functions on the grid for counting a cells neighbors or displaying the grid.

Lua Performance Stats

Grid Size	Average Frame Time	Average Memory Usage KB
12x12	0.000071	50.7306
24x24	0.000234	59.9413
48x48	0.000861	110.4935

Lua Final Thoughts

While I think Lua would be a decent choice for this project, if you plan on running large simulations a more performant language would likely be necessary

Julia

About Julia

Julia is a JIT compiled language similar to JAVA which means that the julia interpreter compiles your julia code into machine code optimized for the system its running on. This is great for performance speed but comes at the cost of memory usage. The julia runtime environment is quite memory hungry needing to load the compiler and standard library into RAM on startup. We see this shown in the performance stats where even the smallest grid is using ~27,000 KB. Thats more than 200 times more memory than Lua uses for its smallest grid. Julia also doesn't garbage collect while the simulation is running unless a specific memory usage threshold is reached to help improve performance when there is still memory available.

Julia Developer Experience

Julia feels like a great balance of performance and ease of use. It provides the benefits of type checked functions without the need to worry about managing memory yourself. The memory overhead is a non issue on most modern hardware and well worth the trade off for the improved performance in this use case.

Julia's array comprehension was also a nice solution for the update_grid function. Instead of using two nested for loops, We use CartesianIndices() to generate a set of coordinates representing every cell in the grid.

Julia Performance Stats

Grid Size	Average Frame Time	Average Memory Usage KB
12x12	0.000023	26765.3746
24x24	0.000070	27601.7965
48x48	0.000286	30545.3284

Julia Final Thoughts

Julia felt easy to program in and provided great performance at the cost high memory usage.

Haskell

About Haskell

Haskell is a functional, statically typed, immutable language compiled using the Glasgow Haskell Compiler. By default expressions are evaluated lazily, or in other words expressions aren't evaluated until their results are needed.

Haskell Developer Experience

Honestly this was some of the least fun I've had coding in a while. Im sure haskell is a great language and their are some good uses for it but I don't think this is one of them. I probably spent at least twice as long as I did on either of the other languages just because the way you need to think about the code is so different. Not to mention all the googling I had to do to figure out how to implement the performance tracking.

Plus after all that work the only main improvement performance wise over julia is the lower memory usage.

Haskell Performance Stats

Grid Size	Average Frame Time	Average Memory Usage KB
12x12	0.000060	132.3747
24x24	0.000172	135.7327
48x48	0.000604	149.2154

Haskell Final Thoughts

While I want to like the language and think the idea of pure functions is cool in theory. Actually

working with them was more effort than help.

Final Recommendation

After working with all three languages and analyzing their performance I would recommend julia as the right tool for the job. It has the best average frame time for all grid sizes and while it also has the greatest memory usage by a large margin. In our use case (Cell Automata), Frame time and developer velocity are more important than trying to be as efficient as possible with our memory.